

Les types de données - Partie 1

Le type undefined

Le type `undefined` est utilisé pour indiquer qu'une variable qu'elle n'a pas de valeur assignée. En JavaScript, si vous déclarez une variable sans lui donner de valeur, elle sera automatiquement définie comme `undefined`.

```
let maVariable; // Déclaration sans valeur
console.log(maVariable); // Affiche: undefined
```

Le type null

Le type `null` est utilisé pour indiquer qu'une variable a été explicitement définie comme n'ayant pas de valeur. C'est une valeur spéciale qui signifie "aucune valeur" ou "valeur vide". Il est souvent utilisé pour initialiser une variable qui sera remplie plus tard ou lorsque vous voulez indiquer qu'une variable n'a pas de valeur significative.

```
let maVariable = 5;

if (maVariable == null) {
    console.log("La variable est nulle.");
} else {
    console.log("La variable a une valeur."); // Affiche: La variable a une
    valeur.
}
```

Les chaînes de caractères (String)

Les chaînes de caractères (ou `strings` en anglais) sont utilisées pour représenter du texte. En JavaScript, une chaîne de caractères est entourée de guillemets doubles ("`...`") ou de backticks (accents graves ("`...`").

```
let sPrenom = "Alice"; // Utilisation de guillemets doubles
let sVille = `Paris`; // Utilisation de backticks
```

Concaténation de chaînes

La concaténation de chaînes consiste à combiner plusieurs chaînes de caractères en une seule. En JavaScript, vous pouvez utiliser l'opérateur `+` pour concaténer des chaînes.

L'avantage des backticks en JavaScript est qu'ils permettent d'inclure des variables à l'intérieur de la chaîne en utilisant la syntaxe `${variable}`. Cela est particulièrement utile pour créer des chaînes dynamiques. Cela

est une fonctionnalité appelée "template literals" ou "gabarits de chaînes" qui est très pratique pour formater des chaînes de caractères de manière lisible.

```
let sNom = "Alice";
let sPrenom = "Dupont";
let sVille = "Paris";

let sMessage1 = "Bonjour, je m'appelle " + sPrenom + " " + sNom + " et j'habite à " + sVille + "."; // Chaîne dynamique avec concaténation et l'opérateur +
console.log(sMessage1); // Affiche: Bonjour, je m'appelle Alice Dupont et j'habite à Paris.

let sMessage2 = `Bonjour, je m'appelle ${sPrenom} ${sNom} et j'habite à ${sVille}.`; // Chaîne dynamique avec backticks
console.log(sMessage2); // Affiche: Bonjour, je m'appelle Alice Dupont et j'habite à Paris.
```

Manipuler les chaînes de caractères

Une fois que vous avez placé une chaîne de caractères dans une variable, vous pouvez accéder à certaines propriétés (caractéristiques) ou utiliser différentes méthodes (actions) prédéfinies pour transformer cette chaîne. Pour cela, vous utilisez le point (.) suivi du nom de la propriété ou de la méthode. La nouvelle valeur résultant de la méthode peut être stockée dans une nouvelle variable ou réassignée à la même variable. Dans ce cas, la variable contiendra la nouvelle valeur après l'application de la méthode et la valeur précédente sera perdue.

Une propriété est une caractéristique d'une variable. Par exemple, la propriété `length` d'une chaîne de caractères contient le nombre de caractères dans cette chaîne. Vous pouvez accéder à une propriété en utilisant le point (.) suivi du nom de la propriété. Exemple, `maVariable.length` vous donnera la longueur de la chaîne stockée dans `maVariable`.

Une méthode est une action que vous pouvez effectuer sur une variable. Par exemple, la méthode `toUpperCase()` convertit tous les caractères d'une chaîne en majuscules. Vous pouvez appeler une méthode en utilisant le point (.) suivi du nom de la méthode et des parenthèses. Exemple, `maVariable.toUpperCase()` convertira la chaîne stockée dans `maVariable` en majuscules. Pour utiliser une méthode, vous devez l'appeler avec des parenthèses, même si elle ne prend pas d'arguments. Si vous oubliez les parenthèses, la méthode ne sera pas exécutée.

Chaque langage de programmation a ses propres méthodes pour manipuler les chaînes de caractères. Si vous apprenez un autre langage, les méthodes peuvent être différentes, mais le principe reste le même.

Voici quelques méthodes courantes lorsque la valeur contenue dans la variable est une chaîne de caractères:

- `length`: Retourne la longueur de la chaîne (le nombre de caractères).
- `toUpperCase()`: Convertit tous les caractères de la chaîne en majuscules.
- `toLowerCase()`: Convertit tous les caractères de la chaîne en minuscules.
- `includes(chaine)`: Vérifie si la chaîne contient une sous-chaîne spécifique (retourne `true` ou `false`).

- `replace(vieilleChaine, nouvelleChaine)`: Remplace une sous-chaîne par une nouvelle sous-chaîne.
- `slice(debut, fin)`: Extrait une partie de la chaîne entre les indices `debut` et `fin`.
- `trim()`: Supprime les espaces blancs au début et à la fin de la chaîne. Pratique lorsque vous récupérez des données utilisateur et que vous voulez éviter les erreurs de saisie.
- `charAt(index)`: Retourne le caractère à une position spécifiée.
- `split(separateur)`: Divise la chaîne en un tableau de sous-chaînes en utilisant le séparateur spécifié.
- `indexOf(chaine)`: Retourne l'indice de la première occurrence d'une sous-chaîne (ou -1 si elle n'est pas trouvée).
- `concat(chaine1, chaine2, ...)`: Fusionne plusieurs chaînes en une seule.
- `startsWith(chaine)`: Vérifie si la chaîne commence par une sous-chaîne spécifique (retourne `true` ou `false`).
- `endsWith(chaine)`: Vérifie si la chaîne se termine par une sous-chaîne spécifique (retourne `true` ou `false`).

Documentation MDN sur les chaînes de caractères

```
let sTexte = "Bonjour le monde!";
let sNom = "Alice";
let nLongueur = sTexte.length; // Utilisation de la méthode length, retourne 18
let sTexteMajuscules = sTexte.toUpperCase(); // Utilisation de la méthode
toUpperCase(), retourne "BONJOUR LE MONDE!"

sNom = sNom.replace("Alice", "Bob"); // Remplace "Alice" par "Bob" et stocke le
résultat dans la même variable
sNom = sNom.toLowerCase(); // Convertit le nom en minuscules et stocke le résultat
dans la même variable

console.log(nLongueur); // Affiche: 18
console.log(sTexteMajuscules); // Affiche: BONJOUR LE MONDE
console.log(sNom); // Affiche: bob
```

Convertir un nombre en chaîne de caractères

Parfois, vous voudrez convertir un élément en texte. Par exemple, si vous avez un nombre et que vous voulez utiliser les méthodes de chaîne de caractères sur ce nombre, vous devez d'abord le convertir en texte. Vous pouvez utiliser la méthode `String()` pour convertir un nombre en chaîne de caractères.

```
let nAge = 30; // Un nombre
let sAge = String(nAge); // Conversion du nombre en chaîne de caractères
console.log(sAge); // Affiche: "30"
console.log(typeof sAge); // Affiche: "string" (type de la variable)
```

```
let nPrix = 19.99; // Un nombre à virgule flottante
let sPrix = String(nPrix); // Conversion du nombre à virgule flottante en chaîne
de caractères
console.log(sPrix); // Affiche: "19.99"
console.log(typeof sPrix); // Affiche: "string" (type de la variable)
```